
[H-1] Storing the password on-chain makes it visible to anyone, and no longer private

Description All data stored on-chain is visible to anyone, and can be read directly from the blockchain. The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword()` function, which is intended to be only called by the owner of the contract.

We can show one such method of reading any data off chain below.

Impact Anyone can read the private password, severely breaking the functionality of the protocol.

Proof of Concept (Proof of Code)

The below test case shows how anyone can read data from the private variable in the code.

1. Create a locally running anvil node:

```
make anvil
```

2. Deploy contract to chain:

```
make deploy
```

3. Run the storage tool

Recommended Mitigation: Due to this, the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts your password.

Likelihood and Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

[H-2] `PasswordStore::setPassword()` has no access controls, meaning a non-owner could change the password.

Description The `PasswordStore::setPassword()` function is set to be an external function, however the natspec of the function and overall purpose of the smart contract is that this function allows only the owner to set a new password.

```
function setPassword(string memory newPassword) external {
    @> // no access controls applied
    s_password = newPassword;
    emit SetNewPassword();
}
```

Impact Any unauthorised user can set/change the password of the contract, severely breaking the contract intended functionality.

Proof of Concept Add the following to the PasswordStore.t.sol test file.

Code

```
function test_non_owner_can_set_password(address randomAddress)
↳ public{
    vm.assume(owner!=randomAddress);
    vm.prank(randomAddress);
    string memory expectedPassword = "myPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

Recommended Mitigation: Add an access control conditional to the setPassword function.

```
if(msg.sender!=s_owner)revert PasswordStore__NotOwner();
```

Likelihood and Impact:

- Impact: HIGH
- Likelihood: HIGH
- Severity: HIGH

[I-1] Wrong Natspec : The PasswordStore::getPassword natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

Description The PasswordStore::getPassword function signature is getPassword while the natspec says it should be getPassword(string).

```
/*
 * @notice This allows only the owner to retrieve the password.
@> * @param newPassword The new password to set.
 */
function getPassword() external view returns (string memory) {
    ...
}
```

Impact The natspec is incorrect.

Recommended Mitigation: Remove the incorrect natspec line.

```
- * @param newPassword The new password to set.
```

Likelyhood and Impact:

- Impact: None
- Likelihood: HIGH
- Severity: Informational/Gas/Non-crits